AI Life Admin Agent

For Messy College Students

Hackathon Build Specification

Complete Feature & Architecture Document

---

1. Project Overview

The AI Life Admin Agent is an autonomous, AI-powered personal assistant built specifically for college students. Its core mission is to eliminate the cognitive overhead of managing academic and personal logistics — so students can focus on learning, not admin work.

The agent runs continuously in the background, reading your emails, tracking your deadlines, managing your calendar, monitoring your grades, and stepping in with motivation when you're overwhelmed. It acts less like a tool you have to use and more like a smart friend who keeps your life from falling apart.

> *"AI forced me to stop procrastinating for 7 days."*
> — Viral angle & core product promise

---

2. Feature Modules

2.1 — Email Reading & Summarization

The email module connects directly to a student's inbox (Gmail/Outlook via OAuth) and continuously monitors incoming messages. Rather than the student having to sift through hundreds of emails, the agent categorizes, prioritizes, and summarizes everything for them in plain English.

Core Capabilities

- OAuth 2.0 integration with Gmail and Outlook/Microsoft 365 — student grants read-only access during onboarding, no password stored
- Automatic email categorization into buckets: Professor/TA, Financial Aid, Campus Admin, Clubs/Organizations, Spam/Promotions, Personal
- AI-generated 1-3 sentence summaries of each email in plain conversational English, stripping legal disclaimers and filler
- Priority scoring from 1–10 based on: sender importance, deadline language detected, urgency keywords, whether the student is in To vs CC

- Action detection — if an email requires a reply, a form submission, or attendance, the agent flags it with a suggested action and due date
- Daily Digest pushed each morning at a configurable time (e.g. 8AM): a ranked list of the top 5 things the student needs to know from their inbox
- Thread summarization — for long back-and-forth threads (e.g. group project coordination), the agent summarizes the full thread state and highlights any open questions directed at the student
- Deadline extraction from email body — if a professor emails "Assignment 3 due Friday at 11:59 PM", the agent parses this and auto-creates a deadline entry (see Section 2.2)

UX Details

- Dashboard panel: Inbox Summary — shows email count by category, today's priority emails, and pending actions
- One-click "Mark Handled" to clear an email from the action queue without leaving the app
- Expandable email cards: click to see the full original email or just read the AI summary
- Smart muting: student can tell the agent "ignore newsletters from X" and it learns to deprioritize or hide those senders

2.2 — Deadline Tracking & Reminders

Deadlines are the single most stressful part of student life. This module creates a centralized, intelligent deadline registry that aggregates from multiple sources — syllabus PDFs, emails, manual entry, and the student's LMS — and sends staged reminders that escalate as the deadline approaches.

Deadline Ingestion Sources

- **Source 1:** Syllabus PDF Parser
  - Student uploads their course syllabi (PDF or image) at semester start
  - Agent uses OCR + LLM extraction to parse all assignments, exams, quizzes, project milestones, and attendance requirements with their dates
  - Confidence score shown for each extracted deadline — student can quickly confirm or correct uncertain ones
- **Source 2:** Email Parsing
  - As described in 2.1, any deadline language detected in emails is auto-added to the deadline registry
  - Duplicates are detected and merged — if a syllabus and an email both mention "Exam 2 on March 14", only one entry is created
- **Source 3:** Manual Entry
  - Student can add any deadline in seconds via a quick-add bar: "Add: Final project demo — April 22 at 2PM"
  - Natural language input supported — agent parses relative dates like "next Thursday" or "in 3 weeks"
- **Source 4:** LMS Integration (Canvas / Blackboard / Moodle)

    - Read-only API integration to pull assignment due dates directly from the student's LMS
    - Syncs every 6 hours to catch any instructor updates

Reminder Escalation Logic

The reminder system is not a simple "remind me 1 day before" setup. It uses escalation logic that adapts to how the student has been behaving:

- Stage 1 — 7 days out: Gentle awareness notification. "Hey, your [Assignment Name] is in a week. You might want to start thinking about it."
- Stage 2 — 3 days out: Planning nudge. "You have [Assignment Name] due in 3 days. Want me to block some study time on your calendar?"
- Stage 3 — 24 hours out: Active alert. Push notification + in-app banner. "Tomorrow at 11:59 PM: [Assignment Name]. Here's what you still need to do."
- Stage 4 — 6 hours out: Urgent escalation. Repeat notifications every 2 hours if student has not confirmed they are working on it.
- Stage 5 — Ignored escalation: If student has dismissed/ignored Stage 3 or 4 reminders more than twice, the agent auto-blocks a study session on their Friday night or next free block (with their prior permission to do this). Shows message: "You've ignored this 3 times — I'm locking in study time for you."
- Repeat offender tracking: The agent remembers which types of assignments the student always leaves to the last minute (e.g., essays) vs. completes early (e.g., quizzes) and adjusts default lead times accordingly

Deadline Dashboard

- Timeline view — horizontal scrollable timeline showing all upcoming deadlines color-coded by course and urgency
- List view — sorted by due date with urgency indicators (green/yellow/orange/red based on time remaining)
- Overdue tracker — if something is past due, it stays on the list marked red so the student can still act (e.g., late submission, talk to professor)
- Completion tracking — student marks deadlines as done; agent tracks completion rate over time as a metric

2.3 — In-Built Calendar System

Rather than just integrating with Google Calendar as a dumb display layer, the agent has an intelligent calendar engine that actively manages the student's time. It knows their class schedule, their habits, their workload, and uses all of that to auto-schedule study blocks and life tasks.

Calendar Data Model

- Fixed events: Class schedule (recurring), recurring commitments like club meetings or part-time job shifts
- Deadlines: Pulled from 2.2 — shown as time-stamped events at the deadline moment

- Study blocks: AI-generated and auto-placed based on deadline proximity and student's historical productivity patterns
- Personal events: Student manually adds events (dentist, birthday, etc.) via natural language
- Sleep schedule: Student sets their typical sleep/wake window; agent never schedules anything during sleep hours

Auto-Scheduling Engine

When a deadline is added or updated, the agent automatically suggests or creates study blocks. The scheduling logic considers:

- Total estimated time needed (student sets this, or agent estimates based on assignment type)
- Available free windows between existing events
- Student's self-reported peak productivity hours (e.g., "I work best 10AM–1PM")
- Buffer time before deadlines — never schedules the final work session within 2 hours of a deadline to reduce panic
- Workload balancing — avoids stacking 4 study sessions on one day if the week overall has capacity
- "Do Not Disturb" periods the student marks as protected personal time

Calendar UX

- Week view as default — shows classes, study blocks, deadlines, and free time at a glance
- Day view — detailed hourly breakdown with task notes inside each block
- Drag-and-drop rescheduling — student can move auto-generated blocks; agent notes the change and updates its model
- Google Calendar / Apple Calendar sync — two-way sync so the student's main calendar stays up to date
- Color coding by type: classes (blue), study blocks (purple), deadlines (red), personal (green)
- Event conflict detection — if a new event conflicts with an existing one, the agent flags it and suggests alternatives

2.4 — Grade Tracking & AI Feedback

The grade tracking module gives students a clear, always-up-to-date picture of where they stand academically — and more importantly, tells them what to do about it. Rather than just displaying numbers, the agent analyzes grade trends and delivers specific, actionable coaching.

Grade Data Entry

- LMS sync (Canvas/Blackboard/Moodle): Pull grades automatically via API; refreshed every 6 hours

- Manual entry: Student enters grades for courses not in a supported LMS; simple form with assignment name, score, weight, and max score
- Grade weight awareness: Agent stores the full grading rubric per course (e.g., "Homework 20%, Midterm 30%, Final 50%") and calculates weighted averages correctly

Grade Dashboard

- Current grade per course: Displayed prominently with letter grade + percentage
- Projected final grade: Based on current performance and remaining assignments/exams, agent calculates the range of possible final grades (best/expected/worst case)
- Grade trend chart: Line graph of grade over the semester — is the student improving, declining, or stable?
- Assignment breakdown: Full list of graded items with color-coded pass/fail/at-risk indicators
- "What do I need?" calculator: Student asks "What do I need on the final to get a B?" — agent computes and answers immediately

AI Feedback & Coaching

This is where the agent goes beyond a gradebook. When grades come in, the agent provides contextual, personalized feedback:

- Post-assignment analysis: "Your quiz scores in CHEM 101 are consistently lower than your homework grades. This suggests you understand the material but struggle with timed recall — consider doing practice quizzes under time pressure."
- At-risk detection: If a student's grade in a course drops below a B- or below their personal target, the agent flags it and suggests specific actions (office hours, tutoring center, study group)
- Effort vs. outcome correlation: Over time, the agent learns which study behaviors correlate with better grades for this student and surfaces that insight
- Professor pattern detection: Notices patterns like "Your lowest grades are always on Friday quizzes — you tend to not review Thursday nights"
- Goal tracking: Student sets target grades at semester start (e.g., "I want at least a B in everything"); agent tracks progress toward those goals and alerts when a goal is at risk
- Encouragement on wins: When a student scores well on a hard exam, the agent acknowledges it specifically and connects it to their effort

2.5 — Motivation, Emotional Support & Goal Reminders

College students don't just struggle with logistics — they struggle with motivation, burnout, and self-doubt. This module gives the agent emotional intelligence. When a student is feeling down, overwhelmed, or checked out, the agent responds with real encouragement grounded in their own personal goals and progress.

Student Goals Onboarding

During setup, the agent conducts a short onboarding conversation where the student articulates:

- Their big-picture goal for college (e.g., "Get into med school", "Land a software engineering internship", "Be the first in my family to graduate")
- Why this goal matters to them — the agent stores a personal "why" statement in first-person, used verbatim in reminders
- What they're afraid of (e.g., "I'm terrified of failing my parents") — stored sensitively and used only when the student has explicitly asked for motivation
- Short-term semester goals (e.g., "Make the Dean's List", "Submit my research paper")
- Personal values or mantras the student wants to be reminded of

Mood Check-In System

- Daily 10-second check-in (optional, student can disable): Agent asks "How are you feeling today?" with a simple emoji/slider scale
- If student selects "Stressed", "Overwhelmed", or "Not Great", the agent immediately offers a supportive response
- If student types a free-text message expressing distress (e.g., "I'm so behind, I want to give up"), the agent detects negative sentiment and switches to support mode
- Student can also manually trigger motivation mode at any time by typing something like "I need motivation" or tapping the "Encourage me" button

Support Mode Behavior

When the agent detects that a student is struggling, it does the following in sequence:

- Acknowledge first — validates how they're feeling without toxic positivity ("That sounds genuinely hard. Juggling 5 courses and feeling behind is exhausting.")
- Remind them of their why — surfaces their personal goal statement: "You told me at the start of the semester that you're doing this because [their words]. That hasn't changed."
- Show evidence of progress — pulls concrete data: "Since September, you've turned in 14 out of 15 assignments. You're more consistent than you feel right now."
- Reframe the immediate situation — breaks the overwhelm into the single next step: "You don't need to fix everything today. The one thing that matters most right now is [highest priority task]."
- Offer a soft reset — suggests a 25-minute focused Pomodoro session, then a break, rather than an overwhelming study marathon
- Resource surfacing — if the student seems persistently distressed over multiple sessions, gently surfaces campus mental health resources (not in an alarming way — framed as "a lot of students find this helpful")

Passive Motivation Features

- Weekly win summary: Every Sunday evening, the agent sends a recap of what the student accomplished that week — deadlines met, study hours logged, grade improvements

- Streak tracking: Tracks how many days in a row the student has logged in, completed tasks, or stuck to their schedule — streak displayed on the home screen
- Motivational micro-messages: Brief contextual encouragement woven into the interface naturally — e.g., "3 days until your exam. You've got time — let's use it."
- Personalized quote rotation: Student can add quotes or affirmations during onboarding that appear on their dashboard home screen, rotating daily

---

3. Agentic Layer — How It Thinks

What separates this from a simple productivity app is the agentic reasoning layer underneath. The agent doesn't just respond to explicit commands — it reasons continuously in the background and takes proactive actions.

3.1 Continuous Background Reasoning

- The agent runs on a periodic reasoning loop (every 15–60 minutes) even when the student is not actively using the app
- At each loop, it checks: any new emails? any deadlines moved closer? any grade updates? any scheduled nudges due? any behavioral patterns worth surfacing?
- Actions taken without prompting: scheduling study blocks, sending reminder notifications, flagging at-risk grades, updating the daily digest
- All proactive actions are logged in an "Agent Activity Log" so the student always knows what the agent did on their behalf

3.2 Memory & Personalization

- Habit memory: Tracks behavioral patterns over time — when the student typically studies, how far in advance they start assignments, which courses they procrastinate on most
- Preference memory: Remembers how the student has customized reminders, dismissed suggestions, or overridden auto-schedules — learns from these signals
- Mood history: Tracks mood check-in data to identify patterns (e.g., always stressed on Tuesdays before morning lectures)
- Grade history: Multi-semester awareness if the student uses the app beyond one semester
- All memory stored locally or in an encrypted user profile — student can view, edit, or delete any stored memory at any time

3.3 Task Prioritization Engine

At any moment, the agent maintains a ranked priority queue of what the student should be doing next. This is computed from:

- Deadline urgency (time remaining + total weight of the assignment)
- Current grade risk (if a course is at a B- and has a major assignment due, it gets elevated priority)
- Ignored reminder count (tasks the student keeps ignoring rise in priority)
- Student-set importance flags (student can mark something as "Critical" manually)
- The #1 priority task is always displayed prominently on the home screen dashboard as "Your Most Important Thing Right Now"

3.4 Escalation Logic

- If a student ignores a high-priority reminder 3 times, the agent escalates from notification to calendar blocking
- If a grade drops into failing territory, the agent escalates from a feedback note to an active intervention conversation
- If a student has not opened the app in 3+ days but has a major deadline approaching, the agent sends an out-of-app push notification as an emergency ping
- Escalation thresholds are configurable — student can set how aggressive they want the agent to be (from "gentle nudges" to "full drill sergeant mode")

---

4. Technical Architecture (Hackathon Build)

4.1 Recommended Stack

Backend

Language	Python (FastAPI) — fast to build, great AI library support
AI Layer	Claude API (claude-sonnet) — email summarization, deadline extraction, feedback generation, motivation responses
Database	PostgreSQL (or SQLite for hackathon speed) — user data, deadlines, grades, memories
Task Queue	Celery + Redis — background reasoning loops, scheduled reminder jobs
Auth	OAuth 2.0 (Google/Microsoft) for email access; JWT for user sessions

Frontend

Framework	React + TypeScript — component-based, good for dashboard UI
Styling	Tailwind CSS — rapid UI development
Calendar	FullCalendar.js — feature-rich calendar component
Charts	Recharts — grade trend graphs, completion rate charts
Notifications	Firebase Cloud Messaging (FCM) — push notifications to mobile/desktop

AI / ML

LLM	Claude API — all natural language tasks (summarization, extraction, generation)
PDF Parsing	PyMuPDF + Claude vision — syllabus extraction
Sentiment Analysis	Claude — mood detection from free-text check-ins
Scheduling Logic	Custom Python algorithm — constraint-based free-slot finder

4.2 Key API Integrations

- Gmail API — read-only email access, label/thread fetching
- Microsoft Graph API — Outlook email access
- Canvas LMS REST API — grade and assignment data
- Google Calendar API — two-way calendar sync
- Firebase Cloud Messaging — cross-platform push notifications

---

5. Data Flows & Context for Parsing

5.1 Core Data Entities

These are the primary data objects the system works with. All AI prompts will reference these entities:

Student Profile

id	UUID
name	String
email_connected	Boolean + provider (gmail/outlook)
semester_goals	Array of goal strings
personal_why	Long text string (their motivation statement)
productivity_peak_hours	Array of time ranges (e.g., ['10:00-13:00', '19:00-22:00'])
sleep_window	{ sleep: '23:00', wake: '08:00' }
escalation_mode	Enum: gentle | standard | aggressive
gpa_target	Float (e.g., 3.5)

Deadline Object

id	UUID
course_id	FK to Course
title	String (e.g., 'Problem Set 4')
due_at	Datetime (ISO 8601)
weight_percent	Float (% of final grade)
estimated_hours	Float (student-set or agent-estimated)
source	Enum: syllabus | email | manual | lms
status	Enum: pending | in_progress | completed | overdue
reminder_stage	Int 1–5 (current escalation stage)
ignored_count	Int

Grade Object

id	UUID
course_id	FK
assignment_title	String
score	Float
max_score	Float
weight_percent	Float
received_at	Datetime
agent_feedback	Text (AI-generated)

Mood Entry

id	UUID
student_id	FK
logged_at	Datetime
mood_score	Int 1–5
free_text	Optional string
agent_response	Text (AI-generated support message)
triggered_support_mode	Boolean

6. Hackathon Demo Script

The following is the recommended 3-minute demo flow that maximizes visual impact for judges:

- **Act 1:** 00:00 — 00:30 | Setup: Show a chaotic inbox with 47 unread emails. Student hasn't checked it in 4 days. A deadline was buried in there.
- **Act 2:** 00:30 — 01:00 | Email Magic: Agent has already read and summarized all 47 emails. Show the priority list. Highlight that a deadline was auto-extracted and added to the tracker.
- **Act 3:** 01:00 — 01:30 | Deadline Cascade: Student clicks the deadline. Shows the escalation — it's been ignored twice already. Agent has auto-blocked a 2-hour study session for tonight. Calendar shows it.
- **Act 4:** 01:30 — 02:00 | Grade Pain: Switch to grades view. CHEM 101 has dropped to a C+. Agent's feedback panel shows: 'You need a 78 on your final to get a B. Based on your current pattern, this is achievable but requires 3 focused review sessions.' Agent offers to schedule them.
- **Act 5:** 02:00 — 02:30 | Motivation Hit: Student types 'I feel like giving up.' Agent responds with their personal why statement, shows them they've completed 14/15 assignments this semester, and suggests one small next step. Emotional moment.
- **Act 6:** 02:30 — 03:00 | Close: Pull back to the home dashboard. Show the 'Most Important Thing Right Now' card, the week view, the grade summary, and the streak counter. Pitch the viral angle.

---

7. Hackathon MVP Scope

Given a 24–48 hour hackathon window, here is the recommended cut of features for a working demo vs. what to mock:

Build Live

- Email reading + AI summarization (Gmail API + Claude)
- Manual deadline entry + reminder logic (at least Stage 1–3)
- Calendar display with auto-study-block suggestion
- Manual grade entry + current grade calculation + one AI feedback message
- Mood check-in + one motivation response using student's stored goal

Mock / Simulate for Demo

- Syllabus PDF parsing (show a pre-processed result, explain how it works)
- LMS grade sync (use hardcoded data to simulate a Canvas pull)
- Multi-stage escalation (demo the Stage 5 calendar blocking as a pre-set scenario)

- Long-term memory and habit modeling (describe as a future feature, show the data model)

Built for students who are trying their best.

The AI doesn't judge you. It just helps you get there.